# Project Summary Paper: Emotion Classification Using Machine Learning

Vilnius University

Faculty of Mathematics and Information Sciences

Data Science Master's Program

By

Edgar Dževeckij

November 2026

Vilnius

# Introduction

## Context

This paper compares how different machine learning frameworks, which focus on NLP applications, match up as emotion classifiers in Lithuanian texts.

## Methodology

The metadata information is sourced from the requirements of a previous project. The metadata information is pre-processed to extract only the text and emotion flag. This text is cleaned and tokenised and later tested on various machine learning models to evaluate how they distinguish different emotions with Lithuanian texts.

## Used Tools

Machine learning models covering different types of feature representation are used: frequency-based representations (TF-IDF), static word embeddings (Word2Vec), and contextual embeddings (Sentence-BERT). Each model is described in their respective segments down below.

# Text Dataset Preparation

## Lithuanian Emotional Speech Corpus (EMO) Metadata

The text samples come from the metadata file of the Lithuanian Emotional Speech Corpus (EMO), which contains almost 350 thousand observations, each a snippet of text that is read aloud in a recording. This experiment uses only the text, so the audio is not relevant. Although the corpus is a diverse source of Lithuanian text, it does not control for several quality factors. A single snippet may be associated with multiple emotions, and the same snippet may appear several times because it was read by different speakers, producing duplicate text records.

## Excluded and Duplicate Texts

Text snippets possessing more than one emotion flag are excluded, so that each observation has a single correct class. Duplicate texts are also removed to prevent test data contamination and overfitting on repeated training observations. The sample sizes before and after cleaning are shown in Table 1, with a detailed breakdown in Appendix A.

**Table 1.**
*Sample Size Reduction During Pre-Processing*

| Data cleaning step | # | % of starting sample |
|---|---|---|
| Starting sample size | 349 696 | 100.0% |
| Multiple emotion observations | 3 548 | 1.0% |
| Remaining sample size after multiple emotion obs. elimination | 346 148 | 99.0% |
| Duplicate observations eliminated | 211 197 | 60.4% |
| Final analysed sample | 134 951 | 38.6% |

*Note*. Compiled by author.

## Emotion Class Distribution

The remaining texts are counted by emotion label to check that each class is balanced and large enough for training without bias. The counts before and after cleaning are shown in Table 2, with a visual comparison in Appendix A.

**Table 2.**
*Sample Size Evolution After Reduction*

| Emotion | Starting, # | Ending, # | Removed, # | Removed, % | Starting share, % | Ending share, % |
|---|---|---|---|---|---|---|
| Anger | 71 978 | 28 057 | 43 921 | 61.0 | 20.8 | 20.8 |
| Joy | 69 174 | 24 997 | 44 177 | 63.9 | 20.0 | 18.5 |
| Neutral | 67 988 | 30 982 | 37 006 | 54.4 | 19.6 | 23.0 |
| Sadness | 70 830 | 26 687 | 44 143 | 62.3 | 20.5 | 19.8 |
| Surprise | 66 178 | 24 228 | 41 950 | 63.4 | 19.1 | 18.0 |

*Note*. Compiled by author.

## Text Analysis

### Text Length

Texts can also be compared by length. Word counts are obtained by splitting each text on spaces, and their descriptive statistics per emotion are shown in Table 3, with additional plots in Appendix B. The statistics are similar across emotions, with an identical minimum, near-identical median, and similar maximum word count. Despite the wide spread between the shortest and longest texts, manual inspection found no data quality issues.

**Table 3.**
*Analysed Texts' Descriptive Statistics*

| Emotion | Texts | Avg word length | Word length std | Min | Median | Max |
|---|---|---|---|---|---|---|
| Anger | 28 057 | 7.38 | 4.29 | 2 | 6 | 27 |
| Joy | 24 997 | 7.14 | 4.07 | 2 | 6 | 25 |
| Neutral | 30 982 | 7.74 | 4.28 | 2 | 7 | 25 |
| Sadness | 26 687 | 6.83 | 3.93 | 2 | 6 | 28 |
| Surprise | 24 228 | 7.04 | 3.88 | 2 | 6 | 25 |

*Note*. Compiled by author.

### Vocabulary Analysis

Lemmatisation normalises the text by reducing each word to its base dictionary form (lemma), so that different forms of the same word are treated as one in the emotion analysis. This reduces the number of unique words and eases computation, while keeping the context consistent. Two tools were compared: spaCy, which is optimised for speed, and Stanza, which is heavier and has more dependencies but is expected to perform better on morphologically rich, non-English languages. Based on the number of resulting lemmas, the reduction ratio and the pass-through rate (Table 4), Stanza appeared the stronger option for normalisation. It reduced the distinct word forms considerably more, whereas spaCy achieved only a modest reduction and returned almost half of the forms unchanged. However, Stanza also made outright errors, such as turning “ačiū” into “atiti” and “nu” into “nūti”, producing words that do not exist. SpaCy, by contrast, passes unrecognised forms through unchanged, so they remain real and only inflate the number of unique lemmas. Since under-lemmatisation is less damaging than introducing spurious lemmas, spaCy was selected for the remainder of the analysis.

**Table 4.**
*Comparison of Different Lemmatisation Algorithms*

| Tool | Word forms processed | Distinct lemmas | Reduction ratio, % | Pass-through rate, % |
|---|---|---|---|---|
| Stanza | 52 814 | 30 549 | 42.16 | 40.85 |
| spaCy | 52 814 | 45 688 | 13.49 | 50.80 |

*Note*. Compiled by author.

To infer possible emotion classes from the lemmas , we can compare the most common words for each emotion class (see Table 5). It is important to mention that before the comparison can be done, stopwords need to be eliminated. SpaCy has a built in stopwords removal tool, so it is used for simplicity purposes. Without using such a tool, it is very plausible that the top words would all just be stopwords. Additionally, using human judgement, additional stopwords are identified and eliminated (see Appendix C). The criterion for identifying additional stopwords is whether they are present in the top 50 most common words for all emotions simultaneously. Using this selection logic, 24 additional words have been eliminated. The table representation can also be visualised as a word cloud, highlighting the most important words. The respective word clouds for each emotion class can be observed in Appendix D.

**Table 5.**
*Most Common Words After Lemmatisation*

| Anger | Joy | Neutral | Sadness | Surprise |
|---|---|---|---|---|
| niekas (782) | džiaugsmas (1 522) | miestas (1 037) | niekas (857) | neįtikėtinti (747) |
| nebūti (545) | širdis (840) | vilnius (899) | liūdna (482) | patikėti (611) |
| reikėti (534) | laimė (713) | lietuva (712) | nebūti (462) | pasaulis (423) |
| negalėti (477) | naujas (636) | nebūti (693) | atrodyti (448) | nebūti (423) |
| sakyti (407) | vaikas (556) | sakyti (691) | liūdesys (420) | negalėti (393) |
| kartas (395) | geras (555) | darbas (665) | negalėti (403) | šimtas (369) |
| darbas (370) | darbas (467) | reikėti (566) | mintis (392) | žemė (362) |
| žodis (346) | dėkoju (456) | teatras (556) | pasaulis (371) | žinoja (348) |
| gerai (335) | meilė (436) | vieta (553) | sakyti (293) | pagalvoti (342) |
| žinoti (325) | galas (389) | tapti (532) | sunkus (280) | įsivaizduoti (309) |

*Note*. Compiled by author.

**Part of Speech (POS) Distributions**

To allow for a better representation of the tokenised words, each word is mapped using the spaCy POS tagging tool. This attaches universal part of speech labels to each lemma analysed. The resulting POS tag distribution, when grouped by emotion class, is provided in Table 6. A detailed visual distribution can be observed in Appendix E. POS tags with a share of at least 5% in at least one emotion class are shown separately, and the remaining tags are grouped under “Other” to reduce visual clutter. Some tags pass the threshold in only some classes, but the main tags (NOUN, VERB, PRON, ADV, ADJ) are shared across all emotions and make up about three quarters of the tokens in each.

**Table 6.**
*Most Common POS Tags Present in the Analysed Sample*

| POS tag | Anger | Joy | Neutral | Sadness | Surprise |
|---|---|---|---|---|---|
| NOUN | 45 946 (22%) | 56 599 (32%) | 82 115 (34%) | 49 511 (27%) | 52 702 (31%) |
| VERB | 41 243 (20%) | 30 400 (17%) | 47 149 (20%) | 37 821 (21%) | 31 876 (19%) |
| PRON | 31 563 (15%) | 15 064 (8%) | 13 481 (6%) | 17 731 (10%) | 14 308 (8%) |
| ADV | 19 831 (10%) | 12 215 (7%) | 16 751 (7%) | 14 805 (8%) | 11 673 (7%) |
| ADJ | 11 626 (6%) | 17 376 (10%) | 19 986 (8%) | 12 721 (7%) | 13 397 (8%) |
| CCONJ | - | 11 246 (6%) | 12 049 (5%) | 9 926 (5%) | - |
| ADP | - | - | - | 9 373 (5%) | - |
| DET | - | 9 128 (5%) | - | - | - |
| PART | 11 195 (5%) | - | - | - | - |
| Other | 45 707 (23%) | 26 352 (15%) | 48 264 (20%) | 30 277 (17%) | 46 620 (27%) |

*Note*. Compiled by author. Rare POS tags (<5%) are grouped as “Other”.

## Text Representation

### Training and Test Sets

To possess representative train and test splits from the analysed sample, stratified splitting is performed for each emotion (resulting in the distribution of each emotion within the two samples being identical). For the current experiment, an 80/20 split is performed, producing the following training and testing samples (see Table 7):

**Table 7.**
*Stratified Train/Test Split Distributions*

| Emotion | Full sample | % of full sample | Train sample | Test sample | Train sample, % | Test sample, % |
|---|---|---|---|---|---|---|
| Anger | 28 057 | 20.8 | 22 445 | 5 612 | 80.0 | 20.0 |
| Joy | 24 997 | 18.5 | 19 998 | 4 999 | 80.0 | 20.0 |
| Neutral | 30 982 | 23.0 | 24 785 | 6 197 | 80.0 | 20.0 |
| Sadness | 26 687 | 19.8 | 21 350 | 5 337 | 80.0 | 20.0 |
| Surprise | 24 228 | 18.0 | 19 382 | 4 846 | 80.0 | 20.0 |
| Total | 134 951 | 100.0 | 107 960 | 26 991 | 80.0 | 20.0 |

*Note*. Compiled by author.

### Frequency-Based Representation

TF-IDF was chosen as a suitable frequency-based representation. Unlike Bag-of-Words, which gives attention based on word frequency, TF-IDF downweighs terms common across most texts and boosts distinctive ones. It is fast and interpretable, as the highest-weighted terms can be inspected directly. Its main limitation is that each word or word pair is an independent feature, so it cannot capture similarity between different unique words or word pairs.

### Static Word Embeddings

Word2Vec was chosen as a suitable static word embedding model. Unlike TF-IDF, it places semantically related words close together in a dense, low-dimensional space, so synonyms are not treated as unrelated features It also perceives contexts based on neighbouring words. Its main limitation is that each word has a single vector regardless of context, so words with several meanings cannot be told apart. This is likely a minor issue for Lithuanian, where inflectional endings often signal the intended meaning.

### Contextual Embeddings

Sentence-BERT was chosen as the contextual embedding model because it does not require fine-tuning, is faster to use, and produces more compact representations than regular BERT (or any other kind of BERT model). Sentence-BERT is already trained to produce one fixed-size vector per text, so the texts can be encoded once and fed directly into a classifier without fine-tuning a large model. Multilingual pretrained versions also support Lithuanian. Its drawbacks include higher computational cost, more implementation dependencies, and lower interpretability.

### Representation Comparison

The three representations differ greatly in size (see Table 8). TF-IDF produces the largest feature space, because every distinct word or common word pair becomes a separate dimension, so its size grows with corpus diversity. Word2Vec and Sentence-BERT instead store fixed length vectors, which is set by model design. Word2Vec learns one vector for each word in the vocabulary while training on the corpus, and a text is represented by the average of the vectors of its words. Sentence-BERT is a pretrained transformer that reads the whole text and encodes it directly into a single vector.

For both models, the default vector sizes are chosen, which is why vectors of Sentence-BERT are somewhat larger than Word2Vec. In both cases the dimensions have no direct meaning on their own, since each one is a learned numerical feature rather than a specific word. The sparsity results support this difference: TF-IDF is almost entirely filled with zeroes, as each input token activates only a fraction of the available vector terms. Word2Vec is almost filled, and Sentence-BERT has no zero values, so every dimension carries information for every text. The difference between the two comes from how each handles unfamiliar words: Word2Vec gives a zero vector to texts made up of extremely rare or misspelled words, while Sentence-BERT encodes any text.

**Table 8.**
*Dimensionality and Sparsity Metrics of Different Machine Learning Models*

| Model | Dimensions | Zero values, % |
|---|---|---|
| TF-IDF | 125 814 | 99.99 |
| Word2Vec | 100 | 0.05 |
| Sentence-BERT | 384 | 0.00 |

*Note*. Compiled by author.

**Semantic Information**

TF-IDF carries the least semantic information, since synonyms and inflected variants are unrelated columns, and word order and negation are only partly recovered through n-grams. For this experiment n-grams include only word pairs of maximum length equal to 2. Word2Vec captures word-level similarity (see Table 9). For frequent words, the nearest neighbours mostly reflect grammatical and contextual similarity, rather than clear synonymy. Some neighbours are unrelated and similarity scores are only moderate, which suggests that the vectors learned from this relatively small. Sentence-BERT, as a pretrained model, is expected to capture sentence-level meaning, including context and negation.

**Table 9.**
*Semantic Information for Most Emotion-Distinct Words*

| džiaugsmas | pyktis | baimė | liūdesys | nuostaba |
|---|---|---|---|---|
| laimė (0.79) | kantrybė (0.76) | fobofobija (0.77) | gyvenimas (0.71) | žalkauskaitė (0.93) |
| neapsakomas (0.74) | ugnis (0.74) | pamišėliai (0.76) | meilė (0.68) | išsiliedavęs (0.92) |
| darbuotojui (0.73) | religija (0.74) | lydeka (0.76) | džiaugsmas (0.67) | alegro (0.92) |
| juokas (0.73) | prakeiksmas (0.73) | karpis (0.75) | egzistencinis (0.67) | pasismauk (0.92) |
| suima (0.71) | baigėsi (0.73) | fobija (0.75) | priešas (0.65) | liūdnoje (0.92) |

*Note*. Compiled by author.

## Emotion Classification

**Classification Results**

Multiclass logistic regression is a suitable method of analysis when the dependent variable is a categorical class value. The final model performance will be evaluated using measures of accuracy, precision, recall, F1 scores, and prediction probability. Visual analysis of the errors using confusion matrices is also completed. The numerical results of the classification exercise, aggregated on model level, is provided in Table 10. More detailed results tables with supporting confusion matrices are provided in Appendix F. TF-IDF achieves the highest accuracy, with the other models falling well behind, even though they capture more meaning. The differentiator may be the relatively low amount of data the models were trained on, compared to the complexity of the Lithuanian language. Word2Vec was trained on the outlined training data for the experiment, while the chosen small Sentence-BERT model already came pretrained.

**Table 10.**
*Classification Evaluation for Different Machine Learning Models*

| Model | Accuracy | Precision | Recall | F1 | True class probability |
|---|---|---|---|---|---|
| TF-IDF | 0.9179 | 0.9197 | 0.9160 | 0.9175 | 0.6585 |
| Word2Vec | 0.4974 | 0.4954 | 0.4898 | 0.4883 | 0.3581 |
| Sentence-BERT | 0.6199 | 0.6216 | 0.6156 | 0.6172 | 0.4995 |

*Note*. Compiled by author.

**Classification Errors**

Given no perfect classification was reached with any model, certain error patterns can help indicate possible model faults which can be improved upon in further research. Aggregating each model's discriminatory power, Table 11 produces the rankings for which pairs of emotions were most often confused. Sadness, surprise, and neutral emotions are present in all error pairs, while joy and surprise were observed less often, indicating a possible pattern for holding more unique vocabulary, which help distinguish those emotions from the rest. Average predicted probabilities show how confidently each model discriminates between different emotions (see Appendix G). Only TF-IDF assigns >60% probability to the true class for every emotion. Sentence-BERT stays between about 44% and 56%. Word2Vec is the least confident, giving the true class as little as 27%, barely above the 17-20% it gives each of the other classes.

**Table 11.**
*Most Common Classification Errors*

| TF-IDF | Word2Vec | Sentence-BERT |
|---|---|---|
| Surprise as Anger (193) | Surprise as Neutral (1 233) | Sadness as Anger (870) |
| Sadness as Anger (185) | Sadness as Neutral (1 223) | Anger as Sadness (766) |
| Surprise as Neutral (175) | Sadness as Anger (1 033) | Surprise as Anger (737) |
| Sadness as Neutral (167) | Surprise as Anger (924) | Surprise as Neutral (722) |
| Joy as Neutral (146) | Joy as Neutral (900) | Sadness as Neutral (722) |

*Note*. Compiled by author.

## Conclusions

From the completed experiments, the following conclusions can be reached:

1. Multiclass logistic regression on three representations of different complexity showed that the simplest one, TF-IDF, performed best, while Word2Vec and Sentence-BERT fell well behind. This may indicate that the training data was too limited for the more complex representations. Performance could improve with using larger models or employing parameter tuning.
2. The more complex representations were also less confident than expected. Their average probability for the true class was close to, or below, the combined probability of all wrong classes, so their aggregated results were not much better than a result observed by a moderate amount of coin flips. TF-IDF was the only model that consistently assigned the true class a clearly higher probability (above 60%).

If the further goal of this experiment is to improve classification performance, two avenues are available: improving data quality and leveraging more advanced model setups. Data quality can be improved with a larger and more diverse training source and more thorough preprocessing, such as more careful lemmatisation and stop word removal, so that the remaining words are the most informative for distinguishing emotions. Model performance can be improved by extending the n-gram range for TF-IDF, tuning the Word2Vec parameters, or using a larger model for Sentence-BERT.

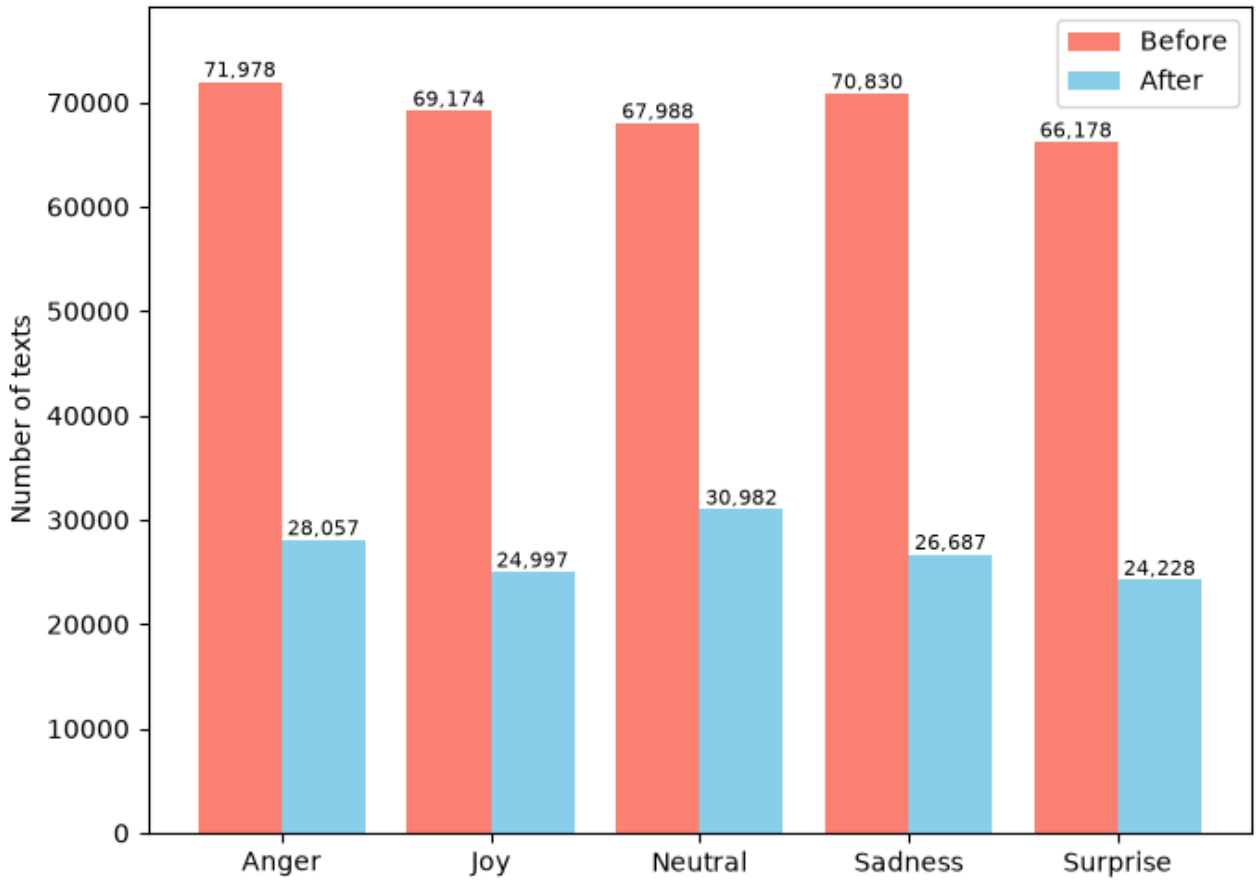
## Appendix A: Sample Size Comparison After Data Preprocessing

**Table 12.**
*Expanded Sample Size Reduction During Pre-Processing*

| Data cleaning step | # | % of starting sample |
|---|---|---|
| Starting sample size | 349 696 | 100.0% |
| Multiple emotion observations | 3 548 | 1.0% |
| Of which classified as “Anger” | 485 | 0.1% |
| Of which classified as “Joy” | 468 | 0.1% |
| Of which classified as “Neutral” | 99 | <0.1% |
| Of which classified as “Sadness” | 571 | 0.2% |
| Of which classified as “Surprise” | 1 925 | 0.6% |
| Remaining sample size after multiple emotion obs. elimination | 346 148 | 99.0% |
| Duplicate observations eliminated | 211 197 | 60.4% |
| Of which classified as “Anger” | 43 921 | 12.6% |
| Of which classified as “Joy” | 44 177 | 12.6% |
| Of which classified as “Neutral” | 37 006 | 10.6% |
| Of which classified as “Sadness” | 44 143 | 12.6% |
| Of which classified as “Surprise” | 41 950 | 12.0% |
| Final analysed sample | 134 951 | 38.6% |

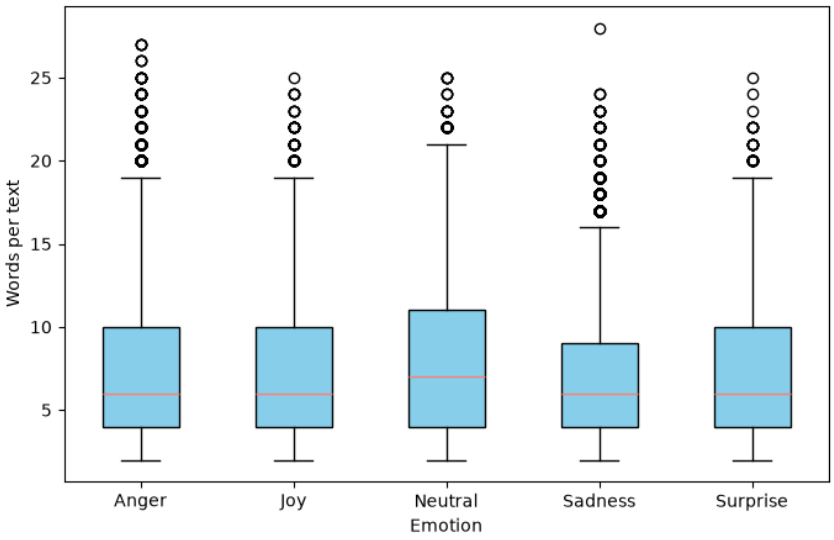
*Note*. Compiled by author.

**Figure 1.**
*Texts per Emotion Before and After Removal*



## Appendix B: Text Word Boxplots by Emotion

**Figure 2.**
*Text Length Distribution per Emotion*




## Appendix C: Stopwords Elimination

**Table 13.**
*Manually Eliminated Stopwords*

| būti | čia | galėti | turėti | būti | čia |
|---|---|---|---|---|---|
| daug | ten | nu | tiek | daug | ten |
| tada | kam | todėl | žmogus | tada | kam |
| dabar | du | trys | eik | dabar | du |

*Note*. Compiled by author. Stopwords are eliminated when they fall within the most common words for all emotions.

## Appendix D: Word Clouds

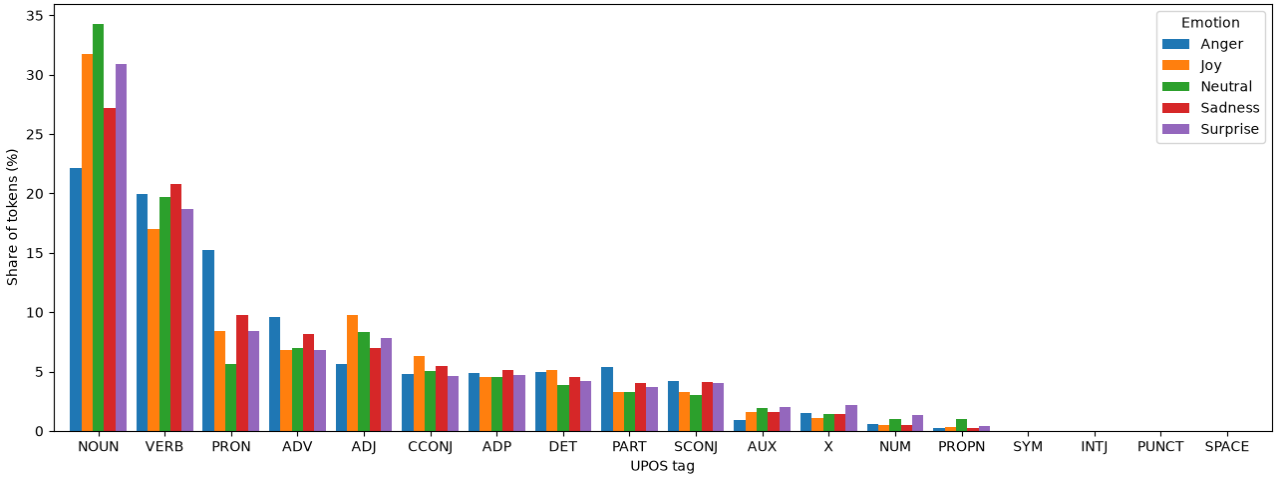
**Figure 3.**
*Word clouds for Different Emotions*



*Note. Compiled by author. Limited to the 50 most common words.*

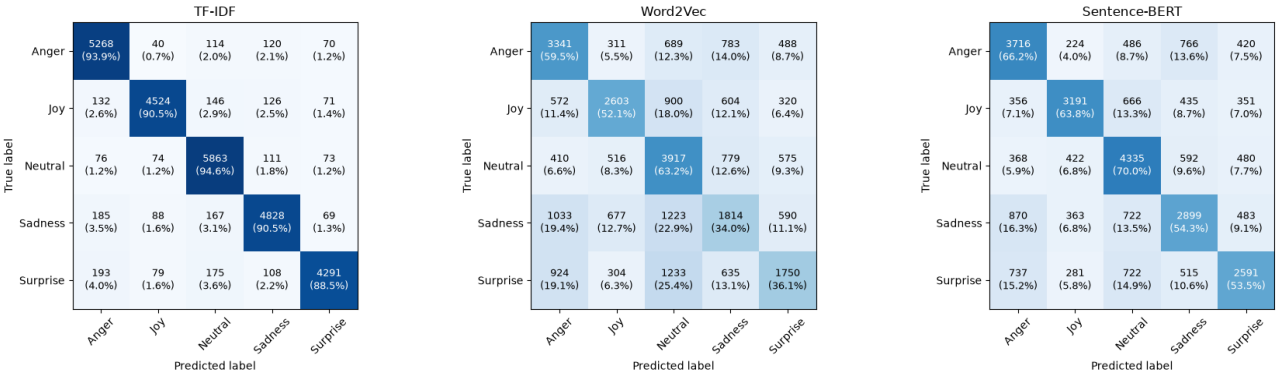
## Appendix E: POS Tag Distribution

**Figure 4.**
*Distribution of POS Tags by Emotion*




*Note*. Compiled by author. Sorted by the most common POS tag in descending order.

## Appendix F: Resulting Confusion Matrices for Each Model Guessing Each Emotion




*Note*. Compiled by author.

## Appendix G: Prediction Probability Comparison for Different Models Guessing Different Emotions

**Table 14.**
*TF-IDF Average Prediction Probability by Emotion*

| Model | Anger | Joy | Neutral | Sadness | Surprise |
|---|---|---|---|---|---|
| Anger | 0.685 | 0.065 | 0.074 | 0.093 | 0.082 |
| Joy | 0.077 | 0.670 | 0.092 | 0.091 | 0.070 |
| Neutral | 0.070 | 0.076 | 0.686 | 0.092 | 0.077 |
| Sadness | 0.101 | 0.086 | 0.104 | 0.624 | 0.086 |
| Surprise | 0.102 | 0.074 | 0.100 | 0.095 | 0.629 |

*Note*. Compiled by author. Header row defines guessed class. Index column defines true class.

**Table 15.**
*Word2Vec Average Prediction Probability by Emotion*

| Model | Anger | Joy | Neutral | Sadness | Surprise |
|---|---|---|---|---|---|
| Anger | 0.406 | 0.114 | 0.127 | 0.193 | 0.160 |
| Joy | 0.132 | 0.418 | 0.167 | 0.171 | 0.114 |
| Neutral | 0.117 | 0.139 | 0.397 | 0.181 | 0.167 |
| Sadness | 0.200 | 0.165 | 0.201 | 0.269 | 0.165 |
| Surprise | 0.188 | 0.117 | 0.215 | 0.179 | 0.302 |

*Note*. Compiled by author. Header row defines guessed class. Index column defines true class.

**Table 16.**
*Sentence BERT Average Prediction Probability by Emotion*

| Model | Anger | Joy | Neutral | Sadness | Surprise |
|---|---|---|---|---|---|
| Anger | 0.517 | 0.075 | 0.102 | 0.173 | 0.133 |
| Joy | 0.086 | 0.560 | 0.140 | 0.114 | 0.100 |
| Neutral | 0.092 | 0.115 | 0.544 | 0.125 | 0.124 |
| Sadness | 0.182 | 0.104 | 0.149 | 0.435 | 0.130 |
| Surprise | 0.158 | 0.100 | 0.161 | 0.138 | 0.442 |

*Note*. Compiled by author. Header row defines guessed class. Index column defines true class.

## Appendix H: Project Source Code

As the project was developed in a Jupyter notebook, allowing outputs to be viewed instantly, the full code cannot be pasted here with its original formatting intact. It is therefore made available at the following location:

https://github.com/3uphori/nlp_course_assignments/blob/main/task_5/report_script_task_5.ipynb

## Appendix I: Project Custom Function List

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, confusion_matrix,
precision_recall_fscore_support


def remove_stop_words(text, lemmas, stop_words):
    return [
        lemma
        for word, lemma in zip(text.split(), lemmas.split())
        if lemma.isalpha() and word not in stop_words and lemma not in stop_words
    ]
```

```
def average_word_vectors(texts, word_vectors):
    vectors = np.zeros((len(texts), word_vectors.vector_size))
    for i, text in enumerate(texts):
        known = [word_vectors[w] for w in text.split() if w in word_vectors]
        if known:
            vectors[i] = np.mean(known, axis=0)
    return vectors


def zero_share(x):
    nonzero = x.nnz if hasattr(x, "nnz") else np.count_nonzero(x)
    return 100 * (1 - nonzero / (x.shape[0] * x.shape[1]))


def evaluate(model, x_test, y_test):
    y_true = np.asarray(y_test)
    y_pred = model.predict(x_test)
    proba = model.predict_proba(x_test)

    classes = list(model.classes_)
    true_proba = proba[np.arange(len(y_true)), [classes.index(c) for c in y_true]]

    precision, recall, f1, support = precision_recall_fscore_support(y_true, y_pred,
labels=classes, zero_division=0)
    cm = confusion_matrix(y_true, y_pred, labels=classes)
    class_accuracy = (len(y_true) - cm.sum(axis=0) - cm.sum(axis=1) + 2 * np.diag(cm))
/ len(y_true)

    report = pd.DataFrame({
        "accuracy": class_accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "true_class_proba": [true_proba[y_true == c].mean() for c in classes],
        "support": support,
    }, index=classes)
    report.loc["macro avg"] = [*report[["accuracy", "precision", "recall", "f1",
"true_class_proba"]].mean(), support.sum()]

    accuracy = accuracy_score(y_true, y_pred)
    report.loc["micro avg"] = [accuracy, accuracy, accuracy, accuracy,
true_proba.mean(), support.sum()]

    mean_proba = pd.DataFrame([proba[y_true == c].mean(axis=0) for c in classes],
index=classes, columns=classes)

    return {
```

```
        "y_pred": y_pred,
        "accuracy": accuracy,
        "report": report,
        "mean_proba": mean_proba,
    }


def plot_confusion_matrix(y_test, y_pred, labels, ax, title):
    counts = confusion_matrix(y_test, y_pred, labels=labels)
    shares = counts / counts.sum(axis=1, keepdims=True)

    ax.imshow(shares, cmap="Blues", vmin=0, vmax=1)
    for i in range(len(labels)):
        for j in range(len(labels)):
            colour = "white" if shares[i, j] > 0.5 else "black"
            ax.text(j, i, f"{counts[i, j]}\n({shares[i, j]:.1%})", ha="center",
va="center", color=colour, fontsize=9)

    ax.set_xticks(range(len(labels)), labels, rotation=45)
    ax.set_yticks(range(len(labels)), labels)
    ax.set_xlabel("Predicted label")
    ax.set_ylabel("True label")
    ax.set_title(title)
```